Agile Methodology

Introduction

Agile is a modern software development methodology that focuses on flexibility, collaboration, customer satisfaction, and continuous improvement. Unlike traditional development models, Agile divides a project into small, manageable iterations called **sprints**, allowing teams to deliver working software frequently and respond quickly to changing requirements.

Agile encourages close collaboration among developers, testers, customers, and stakeholders throughout the software development lifecycle.

What is Agile?

Agile is an iterative and incremental approach to software development where requirements and solutions evolve through collaboration between cross-functional teams and customers.

Instead of delivering the entire project at the end, Agile delivers small functional increments of the software after each sprint.

Objectives of Agile

- Deliver working software frequently.
- Improve customer satisfaction.
- Respond quickly to changing requirements.
- Encourage teamwork and communication.
- Improve software quality.
- Reduce project risks.
- Enable continuous feedback and improvement.

Agile Manifesto

The Agile Manifesto was introduced in **2001** by a group of software developers. It defines four core values that guide Agile software development.

Four Agile Values

1. Individuals and interactions over processes and tools

Effective communication and collaboration among team members are more valuable than relying solely on processes and tools.

2. Working software over comprehensive documentation

Delivering functional software is more important than producing extensive documentation.

3. Customer collaboration over contract negotiation

Customers are involved throughout the project to provide continuous feedback and ensure the product meets their needs.

4. Responding to change over following a plan

Agile welcomes changing requirements, even late in development, to provide greater value to customers.

Agile Principles

The Agile Manifesto is supported by **12 principles**.

1. Customer satisfaction through early and continuous software delivery.
2. Welcome changing requirements.
3. Deliver working software frequently.
4. Business people and developers work together daily.
5. Build projects around motivated individuals.
6. Face-to-face communication is most effective.
7. Working software is the primary measure of progress.
8. Promote sustainable development.
9. Continuous attention to technical excellence.
10. Simplicity is essential.
11. Self-organizing teams produce the best designs.
12. Regular reflection and continuous improvement.

Agile vs Waterfall Methodology

Agile	**Waterfall**
Iterative development	Sequential development
Flexible requirements	Fixed requirements
Continuous customer involvement	Customer involved mainly at the beginning and end
Frequent software releases	Single release after project completion
Easy to adapt to changes	Difficult to change requirements
Continuous testing	Testing after development
Faster feedback	Delayed feedback

Agile	Waterfall
Lower project risk	Higher project risk

Agile Lifecycle

The Agile development lifecycle consists of the following stages:

1. Requirement Gathering
2. Planning
3. Design
4. Development
5. Testing
6. Deployment
7. Review
8. Maintenance

This cycle repeats for every sprint until the project is complete.

Scrum Framework

Scrum is one of the most widely used Agile frameworks. It divides work into short iterations called **sprints**, usually lasting **2–4 weeks**.

Each sprint delivers a potentially shippable product increment.

Scrum Roles

1. Product Owner

Responsibilities:

- Defines product vision.
- Prioritizes Product Backlog.
- Represents customer requirements.
- Accepts completed work.

2. Scrum Master

Responsibilities:

- Facilitates Scrum events.
- Removes obstacles.

- Coaches the team.
- Ensures Scrum practices are followed.

3. Development Team

Responsibilities:

- Design the software.
- Write code.
- Test the application.
- Deliver working software.

The development team is self-organizing and cross-functional.

Scrum Ceremonies

Sprint Planning

The team decides:

- What work will be completed.
- How the work will be completed.

Output:

Sprint Backlog

Daily Scrum (Daily Stand-up)

Duration:

15 minutes

Each team member answers:

- What did I complete yesterday?
- What will I do today?
- Are there any blockers?

Sprint Review

The completed work is demonstrated to stakeholders.

Purpose:

- Collect customer feedback.
- Verify completed features.

Sprint Retrospective

The team discusses:

- What went well?
- What could be improved?
- Action items for the next sprint.

Scrum Artifacts

Product Backlog

A prioritized list of all project requirements maintained by the Product Owner.

Sprint Backlog

A subset of Product Backlog items selected for the current sprint.

Product Increment

The completed and tested functionality delivered at the end of the sprint.

Agile Estimation

Agile estimates work based on **effort** rather than time.

Story Points

Story points measure:

- Complexity
- Risk
- Effort

Common Fibonacci sequence:

1, 2, 3, 5, 8, 13, 21

Example:

Task	Story Points
Login Page	3
Registration	5
Payment Module	13
Search Feature	8

Planning Poker

Planning Poker is a consensus-based estimation technique.

Steps:

1. Product Owner explains the user story.
2. Team discusses the work.
3. Each member selects a story point card.
4. Cards are revealed simultaneously.
5. Differences are discussed.
6. A final estimate is agreed upon.

Benefits:

- Reduces estimation bias.
- Encourages team participation.
- Improves estimate accuracy.

Velocity

Velocity is the number of story points completed in a sprint.

Formula:

Velocity = Total Story Points Completed

Example:

Sprint 1 = 24 points

Sprint 2 = 27 points

Sprint 3 = 29 points

Average Velocity:

(24 + 27 + 29) / 3 = **26.7 story points per sprint**

Velocity helps teams estimate future work and plan upcoming sprints.

Burndown Chart

A Burndown Chart tracks the remaining work during a sprint.

It contains:

- X-axis: Sprint Days
- Y-axis: Remaining Story Points

As work is completed, the remaining story points decrease until they reach zero by the end of the sprint.

Benefits:

- Tracks sprint progress.

- Identifies delays early.
- Helps predict sprint completion.

Sprint Planning Process

The Sprint Planning process includes:

1. Review Product Backlog.
2. Select high-priority user stories.
3. Estimate story points.
4. Break stories into tasks.
5. Assign responsibilities.
6. Create Sprint Backlog.
7. Define Sprint Goal.

The sprint then begins, with the team working collaboratively to complete the selected backlog items.

Iterative Delivery Approach

Instead of delivering the entire project at once, Agile delivers software in small increments after every sprint.

Benefits:

- Faster feedback.
- Reduced risk.
- Early value delivery.
- Continuous improvement.
- Easier maintenance.

User Stories

A User Story describes a feature from the user's perspective.

Standard format:

As a
I want
So that

Example:

As a customer, I want to reset my password so that I can regain access if I forget it.

INVEST Principle

A good user story should satisfy the **INVEST** principle.

Letter	Meaning
I	Independent
N	Negotiable
V	Valuable
E	Estimable
S	Small
T	Testable

Acceptance Criteria

Acceptance Criteria define the conditions that must be met before a user story is considered complete.

The **Given–When–Then** format is commonly used.

Example:

User Story:

As a user, I want to log in so that I can access my account.

Acceptance Criteria:

Given the user is on the login page
When the user enters valid credentials and clicks Login
Then the user should be redirected to the dashboard.

Another example:

Given an invalid password
When the user clicks Login
Then an error message should be displayed.

Collaboration in Agile Teams

Successful Agile teams emphasize collaboration through:

- Daily communication.
- Shared responsibility.
- Continuous feedback.
- Pair programming (when applicable).
- Knowledge sharing.
- Code reviews.
- Retrospectives.
- Cross-functional teamwork.

Every team member contributes to delivering incremental value at the end of each sprint.

Benefits of Agile

- Faster software delivery.
- Higher customer satisfaction.
- Improved collaboration.
- Better software quality.
- Continuous testing.
- Easier adaptation to change.
- Lower development risk.
- Increased transparency.
- Faster issue resolution.
- Continuous improvement.

Challenges of Agile

- Requires experienced teams.
- Frequent customer involvement is necessary.
- Difficult to estimate very large projects.
- Documentation may be less extensive.
- Scope changes can increase complexity if not managed properly.

Real-World Example

Consider a food delivery application.

Instead of building the entire application at once, the team divides the work into multiple sprints:

- **Sprint 1:** User registration and login.
- **Sprint 2:** Restaurant listing and search.
- **Sprint 3:** Cart and order placement.
- **Sprint 4:** Payment integration.
- **Sprint 5:** Order tracking and notifications.

After each sprint, a working increment is demonstrated to stakeholders, who provide feedback. The team incorporates this feedback into subsequent sprints, ensuring the final product aligns with customer needs.

Conclusion

Agile methodology enables organizations to deliver high-quality software through iterative development, continuous feedback, and strong collaboration. Frameworks such as Scrum help teams organize work into manageable sprints, while techniques like story points, planning poker, velocity tracking, burndown charts, user stories, and acceptance criteria improve planning and execution. By

embracing Agile principles and values, teams can adapt to changing requirements, deliver incremental value, and achieve greater customer satisfaction.